C Programming Book

Mohan Arunachalam

Contents

Chapter 1

Introduction to C

- Comparision between English language and C programming language

English Language	C Language
1. Alphabets (`a, b, c, ...`)	1. Keywords (`32`)
2. Works (`More than 10K`)	2. Operators (`45`)
3. Sentance (`Grammer makes this`)	3. Separators (`14`)
	4. Constants

- Constants

Item	value	C datatype
Integer	`5` , `-5` , `0`	int
real	`5.12` , `0.39` , `5.00` , `-3.25`	float
charater	`'a'` , `'5'`	char

- Reason for the name float but why not real?
  - 100. `231452156`
    - `.` is decimal point
    - `231452156` : This keep on changing So `float`
- Primary / Primitive / Fundamental / Basic data types in C
  - `Data` is a meaningfull information
  - `Datatype` is a different kind of data like `int` , `float` and `Char`

Format Specifier	Data Type	Description
%c	char	Prints a single character
%s	char *	Prints a string
%d	int	Signed decimal integer
%i	int	Signed integer (same as `%d` in `printf`)
%u	unsigned int	Unsigned decimal integer
%hd	short int	Signed short integer
%hu	unsigned short int	Unsigned short integer
%ld	long int	Signed long integer
%lu	unsigned long int	Unsigned long integer
%lld	long long int	Signed long long integer
%llu	unsigned long long int	Unsigned long long integer
%f	float	Floating-point number (`printf` promotes float to double)
%lf	double	Double-precision floating-point (`scanf` uses `%lf` for double)
%Lf	long double	Long double floating-point

Format Specifier	Data Type	Description
%e / %E	float, double	Scientific notation
%g / %G	float, double	Shortest representation of `%f` or `%e`
%x	unsigned int	Hexadecimal integer (lowercase)
%X	unsigned int	Hexadecimal integer (uppercase)
%o	unsigned int	Octal integer
%p	void *	Pointer address
%%	—	Prints `%` character
%zu	size_t	Unsigned size type
%td	ptrdiff_t	Pointer difference type
%jd	intmax_t	Largest signed integer type
%ju	uintmax_t	Largest unsigned integer type

1.1 List of all Operators

Operator	*Description*	*Associativity*
() **[]** **.** **->** **++ --**	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
++ -- **+ -** **! ~** **(type)** ***** **&** **sizeof**	Prefix increment/decrement Unary plus and minus not operator and bitwise complement type cast Indirection or dereference operator Address of operator Determine size in bytes	right to left
*** / %**	Multiplication, division and modulus	left to right
+ -	Addition and subtraction	left to right
<< >>	Bitwise left shift and right shift	left to right
< <= **> >=**	relational less than/less than equal to relational greater than/greater than or equal to	left to right
== !=	Relational equal to or not equal to	left to right
&&	Bitwise AND	left to right
^	Bitwise exclusive OR	left to right
|	Bitwise inclusive OR	left to right
&&	Logical AND	left to right
||	Logical OR	left to right
? :	Ternary operator	right to left
= **+= -=** ***= /=** **%= &=** **^= |=** **<<= >>=**	Assignment operator Addition/subtraction assignment Multiplication/division assignment Modulus and bitwise assignment Bitwise exclusive/inclusive OR assignment	right to left
,	comma operator	left to right

Figure 1.1: Operators

1.2 Assignment Operator

- It requires 2 arguments
- Left side arg must be variable
- Right side arg can be a variable / constant / expression
- Left side value is called l-value

```
// Not valid
a = 5
a = ;
  = 10;
10 = 20;

// Valid
a = 5;
b = a;
c = 2 + 3;
```

- Every expression is replaced with constant

```
a = 5;
b = -3;
c = 9;

a = b;  // a = -3
b = -c; // b = -9
a = -b; // a = 9

-c = a; // Error because l-value is expresion not a variable
```

- In C language each and every statement ends with `;`
- Semicolon is called statement terminator (or) end of statement
- Why `.` is not used as terminator?
  - because `.` is used as decimal point in float number
- What is l-value error?
  - On the left hand side of assignment operation we should provide a variable but by mistake if we provide constant (or) expression then we get l-value required error.

1.3 Arithmetic Operators

```
// Example 1: Expression evaluation
a = 2 + 3;
a = 2 + 3 + 4;
a = 6 - 2 + 3 - 4 + 1;
a = 2 + 3 * 5;
a = (2 + 3) * 5;
a = 2 * 3 + 3 * 2;
```

Operation on	Result
int with int	int
int with float	float
float with int	float
float with float	float

```
// Example 2:
a = 5 / 2;   // 2
a = -5 / 2;  // -2
a = 5 / -2   // -2
```

```
a = -5 / -2  // -2

a = 5.0 / 2; // 2.5
a = 5 / 2.0; // 2.5
a = -5 / 2.0 // -2.5
a = 2 / 5    // 0
a = 2 / -5   // 0
```

```
// Example 3:

a = 17 / 3 / 2;
/*
a = 5 / 2
a = 2
*/

a = 15 / 2 * 3;
/*
a = 7 * 3
a = 21
*/

a = 9 * / 4;
/*
a = 27 / 4
a = 6
*/

a = 6 * 3 / 4 * 5;
/*
a = 18 / 4 * 5
a = 4 * 5
a = 20
*/
```

Division	Result of division	Modulus	Result of modulus
+ / +	+	+ % +	+
+ / -	-	+ % -	+
- / +	-	- % +	-
- / -	+	- % -	-

- **Floating point numbers cannot used in modulus only integer is accecptable**

```
// Example 1

a = 5 / -2; // -2
a = 5 % -2; // 1
```

```
// Example 2

a = 3.5 * 2 % 7; // Compilation error
```

- The result of modulus operation is remainder
- Result sign is same as numerator sign
  - If numerator is +ve then result is +ve
- Result sign does not depend on denominator
- `Condition 1` : x % y == 0 means then we can say that `x` is multiple of `y`
- `Condition 2` : Numerator % 10 givers last digit of numerator

- `Condition 3` : When numerator is smaller then denominator then result is numerator
- `Condition 4` : When numerator is equal to denominator then result is zero

1.4 C-Tokens

- Smallest individual unit in C language is called C-Token
- C-Token are
  - Keywords
  - Operators
  - Separators
  - Constants
  - Identifiers
- There can be 'N' no. of space / Tabs / Newlines between two C-Tokens
- Program should be in good indentation

```c
void main()
{
    printf("Hello");
}
```

- The above program has following C-Tokens
  - Keyword : `void`
  - Operator : `(` `)` `(` `)`
  - Separator : `{` `;` `}`
  - Constant : `"Hello"`
  - Identifiers : `main` `printf`
- Total `11` C-Tokens are there in above program

1.5 printf function

- It will print 1st arg on screen
- 1st arg must be in pair of double quotes
- if more than one arg, then evey arg must be separated by commas

```c
printf("Hello");
// Hello

printf("    Hello    ");
// ....Hello....
// (Spaces are retained)

printf("Hello%dadc%d", 10, 20);
// Hello10abc20

printf("%d %d %d", 10, 20, 30);
// 10 20 30

printf("%d%d%d", 10, 20, 30);
// 102030

printf("%d,%d,%d", 10, 20, 30);
// 10,20,30

printf("%d %d %d", 10, 20);
// 10 20 <GV/Junk>

printf("%d %d %d", 10);
```

```c
// 10 GV GV

printf("%d %d %d");
// GV GV GV

printf("%d %d %d", 10, 20, 30, 40);
// 10 20 30

printf("%d %f %c", 10, 3.75, 'a');
// 10 3.75 a

printf("%d", 5.5);
// GV

printf("%f", 5);
// GV

printf("%d", 5 + 2);
// 7

printf("5 + 2");
// 5 + 2

printf("%d + %d", 5 + 2);
// 7 + GV

printf("%d * %d = %d", 5, 2, 5+2);
// 5 * 2 = 7

printf("%d * %d = %d", 5, 2, 5*2);
// 5 * 2 = 10

printf("%f", 5/2);
// GV

printf("%d", 5/2);
// 2

printf("%f", 5.0 % 2);
// Error

printf("%d", -5 % -2);
// -1

printf("Hello", "Hai", "Bye");
// Hello

printf("Hello""Hai""Bye");
// HelloHaiBye

printf(""Hello"");
// Error

printf("""Hello""");
// Hello

printf("""Hello""");
//Error
```

```

printf("%d %d %d", "%d %d", 10, 20, 30, 40, 50);
// GV 10 20

printf("10, 20", "%d");
// 10, 20

int ts = 7500;
printf("ts");                  // ts
printf(ts);                    // Error
printf("Total sal = %d", ts); // Total sal = 7500
printf("%d + 1000", ts);       // 7500 + 1000
printf("%d", ts + 1000);       // 8500
```

1.6 Variable definition Rules

- Variable definition ways,

```
// Method 1:
int a;
int b;
int c;

// Method 2:
int a, b, c;
```

- In the above two style 1st style is recommanded because,
  - Subsequent change in variable type in future is easy
  - Writing and updating a comment is easy
- ***Variable definition***: Compier allocates memory for that variable

```
int a;
int b = 10;
```

- ***Variable declaration***: Compier doesn't allocates memory for that variable

```
extern int a;
```

Exmaples:

```
void main()
{
    int a; // Always say datatype to a variable
    a = 5;
    printf("%d", a);
}

// Output
// 5

void main()
{
    int i;
    float f;
    i = 5;
    f = 5.5;
    printf("%d %f", i, f);
}

// Output
```

```
// 5 5.5
```

- In C programming we can assign any type of data to any type of variable. Internally in computer the compiler will take care.
- In printf function only sutiable data is given to format specifier else garbage value is printed.

```
void main()
{
    int i;
    i = 5.5;
    printf("%d", i); // 5
}

void main()
{
    float f;
    f = 5;
    printf("%f", f); // 5.0
}

void main()
{
    int i;
    i = 5 / 2;
    printf("%d", i); // 2
}

void main()
{
    float f;
    f = 5 / 2;
    printf("%f", f); // 2.0
}

void main()
{
    int i;
    i = 5.0 / 2;
    printf("%d", i); // 2
}

void main()
{
    float f;
    f = 5.0 / 2;
    printf("%f", f); // 2.5
}
```

- More examples

```
/************************** Valid definition *********************************/

int a;
a = 5; // Assignment

int a = 5; // Initialization

int a = 18 / 3 / 5;

```

```c
int a = 2 + 5;

int a = 7.65;

int a, b, c;
a = b = c = 10; // Valid

int a, b, c = a = b = 10;

int a = a;

int a1, b1, c1;

int avgofsallary; // Valid but not readable

int avg_of_salary;

int a, A;

char way2ms;

char INT;

int abcdefghijklmnopqrst;

int _;

char _1, _2;

/****************************** Not a valid *********************************/
int a = b = c = 10;

int 1a, 1b, 1c;

int avg of salary; // Not a valid. No white space allowed

int if;

int a + b;

int a, a; // Not a valid. same name not allowed

float 1606y2;

int 42shared;

char `a`, `b`, `c`;
```

- Rules for naming a valid variable Names:
  - An Identifier can contain
    - a - z
    - A - Z
    - 0 - 9
    - _ (underscore)
  - An Identifier name start with alphabet (or) underscore
  - No whitespace, operator are allowed
  - No keyword, but act as a pert of variable
    - Example: `int intfloat`

- No restriction in length of variable name, but if length is very big then readability is less. Hence programmers recommandation is max of 15 characters

```
int pwm_analog_read;
int pwm_analog_write;
int pwm_digital_read;
int pwm_digital_write;
```

1.7 Relational Operators

Operator	Name	Description
`==`	Equal to	Checks whether two operands are equal
`!=`	Not equal to	Checks whether two operands are not equal
`>`	Greater than	Checks whether left operand is greater than right operand
`<`	Less than	Checks whether left operand is less than right operand
`>=`	Greater than or equal to	Checks whether left operand is greater than or equal to right operand
`<=`	Less than or equal to	Checks whether left operand is less than or equal to right operand

```
a = 5 + 2;   // a is int

b = 1.5 * 3; // b is float

c = 4 > 3;   // c and d is boolean (but not available in C)
d = 9 < 7;
```

- The use of this operator
  - To establish relation between two numbers and perform comparition
  - Result of relation operation is either `true` or `false`
  - `true` and `false` are boolean datatype but C does not support that hence
    - `true` = Non-zero
    - `false` = zero

```
void main()
{
    int a;
    a = 4 > 3 > 2;
    printf("%d", a); // 0
}
```

- In Maths,
  - a > b and b > C makes a > c. So a > b > c
  - 4 > 3 and 3 > 2. So 4 > 3 > 2
- In English, Lets take below two scentance
  - I am watching movie
  - I am eating popcorn
  - Now, Lets combine two statement blindly
    - I am watching movie eating popcorn `(meaning less)`
  - So, I am watching movie `and` eating popcorn `(meaningful now)`
- In C Language
  - `4 > 3 > 2` makes result in `0` even if we think in matematically and gramatically
  - So, `(4 > 3) && (3 > 2)` makes result `1`

```
a = 4 + 3 > 2 + 5;
  = 7 > 2 + 5
  = 7 > 7
```

```
  = 0

a = 6 > 3 + 2 < 8
  = 6 > 5 < 8
  = 1 < 8
  = 1

a = 6 * 3 >= 5 * 2 + 8 >= 17 + 8 <= 6 * 4 == 8 * 5 != 5 * 3 * 2 + 10 != 40;
// Based on the Associativity of operator all are executed from left to right
// and last operator is !=. So result always will be 1 or 0
```

1.8 Logical Operator

- If we are combining 2 or more relation operator directly then result is unexpected
- To overcome this C people have introduced logical operators
- Thus use of logical operation is to combine two (or) more relational statement and to get compound statement

Operator	Name	Description
&&	Logical AND	Returns true if both conditions are true
||	Logical OR	Returns true if at least one condition is true
!	Logical NOT	Reverses the logical state of operand

A	B	A && B	A || B	!A	!B
0	0	0	0	1	1
0	1	0	1	1	0
1	0	0	1	0	1
1	1	1	1	0	0

- Whenever we are having 'N' condition and if we are depend on all 'N' condition at that time we are going for `AND` logic
- When there are 'N' condition and if we are depend on any 1 condition that time we gone for `OR` logic
- `NOT` is used in negative test condition

Chapter 2

Flow control

2.1 if Conditional Construct

```
// (1)

if( condition ) {
    // (2) if body;
}

// (3)
```

- If condition is `True` then (1), (2) and (3) will execute.
- If condition is `False` then (1) and (2) will execute.

```
    void main()
    {
        printf("A");
        printf("B");

        if(3 > 2) {
            printf("C");
            printf("D");
        }

        printf("E");
        printf("F");
    }

    // Output
    // ABCDEF
```

```
    void main()
    {
        printf("A");
        printf("B");

        if(3 > 20) {
            printf("C");
            printf("D");
        }

        printf("E");
        printf("F");
    }

    // Output
    // ABEF
```

- If there is no `if` body / `else` body then compier will assume the first semicolon as a part of `if` body

```c
void main()
{
    printf("A");
    printf("B");
    if(3 > 2)
    printf("C");
    printf("D");
    printf("E");
    printf("F");
}

// Output
// ABCDEF
```

```c
void main()
{
    printf("A");
    printf("B");
    if(3 > 20)
    printf("C");
    printf("D");
    printf("E");
    printf("F");
}

// Output
// ABDEF
```

- In C programing `;` alone is valid.
- The purpose of the `;` is to provide a delay to next instruction.
- The line that contains only `;` is called Null statement, Empty statement, Dummy statement.

```c
// Valid program
void main()
{
    ;
}
```

```c

void main()
{
    printf("A");
    printf("B");
    if(3 > 2);
    printf("C");
    printf("D");
    printf("E");
    printf("F");
}

// Output
// ABCDEF
```

```c

void main()
{
    printf("A");
    printf("B");
    if(3 > 20);
    printf("C");
    printf("D");
    printf("E");
    printf("F");
}

// Output
// ABCDEF
```

```c
void main()
{
    printf("A");
    printf("B");
    if(3 > 2);        // Compier will execute this with meaningless
    {                 // Dummy body start
        printf("C");
        printf("D");
    }                 // Dummy body end
    printf("E");
    printf("F");
}

```

```
// Output
// ABCDEF
```

- In C programming, Non-zero values are consider as True and zero is consider as False

---

```

void main()
{
    printf("A");
    printf("B");
    if(20)
    {
        printf("C");
        printf("D");
    }
    printf("E");
    printf("F");
}

// Output
// ABCDEF
```

```

void main()
{
    printf("A");
    printf("B");
    if(0)
    {
        printf("C");
        printf("D");
    }
    printf("E");
    printf("F");
}

// Output
// ABEF
```

```

void main()
{
    printf("A");
    printf("B");
    if( )
    {
        printf("C");
        printf("D");
    }
    printf("E");
    printf("F");
}

// Output
// Compilation Error
```

- Interview based program

```
void main()
{
    int a = 10;
    int b = 20;
    int max;

    /* To find max of two integer using if */
    if(a > b) {
        max = a;
    }

    if(b > a) {
        max = b
    }

    /* WAP to find max of 2 integer using only one if */
    max = b;
    if(a > b) {
        max = a;
    }

    /* WAP to find max of 2 integer using 1 if and without 3rd variable */
    if(a > b) {
        b = a;
    }
    printf("MAX = %d", b);

    /* WAP to find max of 2 integer without using any if without 3rd variable */
    max = (a > b) * a + (b < a) * b;

    /* WAP to find max of 2 integer without using any condition */
    max = (a + b + abs(a - b)) / 2;
```

```
}
```

2.2 if-else Conditional Construct

```
// (1)

if( condition ) {
    // (2) if body
} else {
    // (3) else body
}

// (4)
```

- If condition is `True` then (1), (2) and (4) will execute.
- If condition is `False` then (1), (3) and (4) will execute.
- Whenever conditon is true do some specific job. When condition is false do some other specific task. At that time we are going for `if-else` conditional construct.
- `else` keyword must be immediatly placed after `if` body.
- If some statement is placed between `if` body and `else` then that is error.

```
void main()
{
    printf("A");
    printf("B");
    if(3 > 2)
    printf("C");
    printf("D");
    else
    printf("E");
    printf("F");
    printf("G");
    printf("H");
}

// Output
// Compilation Error
```

Program to find max of 2 integers

```
if(a > b) {
    max = a;
} else {
    max = b;
}
```

Program to find max of 3 integers

```
if((a > b) && (a > c)){
    max = a;
} else {
    if(b > c) {
        max = b;
    } else {
        max = c;
    }
}
```

```
if(a > b) {
    if(a > c) {
        max = a;
    } else {
        max = c;
    }
} else {
    if(b > c) {
        max = b;
    } else {
        max = c;
    }
}
```

Examples

```

void main() {
    int a = 10;
    if(a == 5) {
        printf("Hello%d", a);
    } else {
        printf("Hai%d", a);
    }
}

// Output
// Hai5
```

```

void main() {
    int a = 10;
    if(a == 10) {
        printf("Hello%d", a);
    } else {
        printf("Hai%d", a);
    }
}

// Output
// Hello10
```

- Assignment operation has 2 actions
  - Assign value to variable
  - Replace the expression with assigned value
- Why this behaviour?

```
int a1, b1, c1;
c1 = b1 = a1 = 10; // This results in all variables assigned to value 10

int a2;
float b2;

a2 = b2 = 3.5 // a2 = 3, b2 = 3.5
b2 = a2 = 3.5 // a2 = 3, b2 = 3.0
```

```
void main() {
  int a = 10;
  if(a = 0) {
    printf("Hello%d", a);
  } else {
    printf("Hai%d", a);
  }
}

// Output
// Hai0
```

```
void main() {
  int a = 10;
  if(a = 1) {
    printf("Hello%d", a);
  } else {
    printf("Hai%d", a);
  }
}

// Output
// Hello1
```

```
void main() {
  int a = 10;
  if(a = 0.75) {
    printf("Hello%d", a);
  } else {
    printf("Hai%d", a);
  }
}

// Output
// Hai0
```

More analysis

```
// What if the below conditions are in if condition?

a == 10;   // Hello10
a == 5;    // Hai10
a == 1;    // Hai10
a == 0;    // Hai10
a == 10.5; // Hai10
a == 10.0; // Hello10
a == 0.75; // Hai10

a = 10;   // Hello10
a = 5;    // Hello5
a = 1;    // Hello1
a = 0;    // Hai10
a = 10.5; // Hello10
a = 10.0; // Hello10
a = 0.75; // Hai10
```

```
a = 10;
b = 20;
c = a && b = 30;

printf("%d %d %d", a, b, c);

// Output
// Compilation Error: l-value required
```

```
a = 10;
b = 20;
c = a && (b = 30);

printf("%d %d %d", a, b, c);

// Output
// 10 30 1
```

Chapter 3

Bitwise Operators

Operator	Name	Description
&	Bitwise AND	Sets bit to 1 if both bits are 1
|	Bitwise OR	Sets bit to 1 if at least one bit is 1
^	Bitwise XOR	Sets bit to 1 if bits are different
~	Bitwise NOT / One's Complement	Inverts all bits
«	Left Shift	Shifts bits to left
»	Right Shift	Shifts bits to right

3.1 One's Complement

!	~
Output is 1 or 0	output is integer
!3.14 is 0	~3.14 is error

```c
void main()
{
    int a;
    a = ~0;
    printf("%d", a);
}

/*
if int datatype = 2 bytes

0  = 0000 0000 0000 0000 (result = 0)
~0 = 1111 1111 1111 1111 (result = -1)
*/
```

- If MSB = 0, then number is +ve and it is original data
- If MSB = 1, then number is -ve and number is in 2's complement.

3.1.1 Binary(2's complement) to decimal

- If MSB = 0, Convert normally from binary to decimal.
- If MSB = 1,
    1. Take 2's complement again
    2. Convert to decimal
    3. Add negative sign

Example

```
/*
Binary(2's complement) = 1111 1111 1111 0101
original data          = 0000 0000 0000 1010
                                        +1
                        --------------------
                         0000 0000 0000 1011 = 11 (consider this as negative)
*/
```

```
/*
~5
~(0000 0000 0000 0101)
1111 1111 1111 1010
                | |
                V V
                4+1 = 5
                      |
                      V
                      5 + 1 = 6 (consider as negative)

~5 = -6
*/
```

```
/*
~9
~(0000 0000 0000 1001)
1111 1111 1111 0110
               |  |
               V  V
               8 +1 = 9
                      |
                      V
                      9 + 1 = 10 (consider as negative)

~9 = -10

So,
~0 = -1
~5 = -6
~9 = -10
~37 = -38
~-6 = 5
~-3107 = 3106
~-10906 = 10905
~3.14 = Error (Complement on float is error)
*/
```

3.1.2 Summary

1. Add 1
2. Change the sign of result

3.2 Left shift Operator

```
/*
5 << 1 = 0000 0000 0000 0101
         0000 0000 0000 1010 (shift by 1) = 10

5 << 2 = 0000 0000 0000 0101
         0000 0000 0001 0100 (shift by 2) = 20

-5 << 1 = 1111 1111 1111 1011
          1111 1111 1111 0110 (shift by 1) = -10

-6 << 2 = 1111 1111 1111 1010
          1111 1111 1110 1000 (shift by 2) = -24
*/
```

3.2.1 Summary

a « b is equal to a * 2^b

3.3 Right shift Operator

```
/*
5 >> 1 = 0000 0000 0000 0101
         0000 0000 0000 0010 = 2
         |
         V
         This position is filled by the value of MSB because signed

11 >> 2 = 0000 0000 0000 1011
          0000 0000 0000 0010 = 2
          ||
          VV
          This position is filled by the value of MSB because signed

-9 >> 1 = 1111 1111 1111 0111
          1111 1111 1111 1011 = -5
          |
          V
          This position is filled by the value of MSB because signed

-6 >> 2 = 1111 1111 1111 1010
          1111 1111 1111 1110 = -2
          ||
          VV
          This position is filled by the value of MSB because signed
*/
```

3.3.1 Summary

1. $$5 >> 1 = \frac{5}{2^2} = 2$$

2. $$11 >> 2 = \frac{11}{2^2} = 2$$

3. $$-9 >> 1 = \sim (\frac{\sim -9}{2^1}) = \sim (\frac{8}{2}) = \sim (4) = -5$$

4. $$-6 >> 2 = \sim (\frac{\sim -6}{2^2}) = \sim (\frac{5}{4}) = \sim (1) = -2$$

$$a >> b = \begin{cases} \frac{a}{2^b}, & \text{if } a \text{ is positive} \\ \left(\sim \frac{\sim a}{2^b}\right) & \text{if } a \text{ is negative} \end{cases}$$

- For signed data gap is filled with MSB
- For unsigned data gap is filled with 0
- Shifting like `10 << 40` , `81 >> 100` , `10 << -2` , `8 >> -3` are machine dependant

3.4 Bitwise AND, OR, XOR

AND	XOR	OR
x & 1 => x	x ^ 1 => !X **(Toggle)**	X | 1 => 1 **(Set)**
x & 0 => 0 **(Clear)**	X ^ 0 => X	X | 0 => X

- `X` is don't care

3.4.1 Summary

1. Set a bit, use OR
2. Clear a bit, use AND
3. Toggle a bit, use XOR

3.5 Problem solving

3.5.1 Check if the nth bit is set

```
bool is_bit_set(uint32_t x, uint8_t n)
{
    return (x >> n) & 1U;
    /* OR: return (x & (1U << n)) != 0; */
}
```

3.5.2 Set the nth bit

```
uint32_t set_bit(uint32_t x, uint8_t n)
{
    return x | (1U << n);
}
```

3.5.3 Clear the nth bit

```c
uint32_t clear_bit(uint32_t x, uint8_t n)
{
    return x & ~(1U << n);
}
```

3.5.4 Toggle the nth bit

```c
uint32_t toggle_bit(uint32_t x, uint8_t n)
{
    return x ^ (1U << n);
}
```

3.5.5 Count number of set bits

```c
uint8_t count_set_bits(uint32_t x)
{
    uint8_t count = 0;
    while (x) {             /* loop runs exactly as many times as there are set bits */
        x &= x - 1;        /* x-1 flips all bits from LSB up to (and including) the
                              lowest set bit, so AND-ing zeroes that bit out.
                              e.g.  x     = 1010 1000
                                    x-1   = 1010 0111
                                    x&x-1 = 1010 0000  <- one set bit gone */
        count++;
    }
    return count;           /* result = number of 1-bits in original x */
}
```

3.5.6 Check if a number is a power of 2

```c
bool is_power_of_2(uint32_t n)
{
    /* n=0 excluded: 0 is not a power of 2 */
    return (n != 0) && ((n & (n - 1)) == 0);
}
```

3.5.7 Swap two numbers using XOR

```c
void xor_swap(uint32_t *a, uint32_t *b)
{
    if (a != b) {          /* guard against aliasing */
        *a ^= *b;
        *b ^= *a;          /* *b = original *a */
        *a ^= *b;          /* *a = original *b */
    }
}
```

3.5.8 Find first set bit in a number

```c
/* Shift right until LSB is 1, counting each step.
 * O(position of lowest set bit) — worst case O(32).              */
int first_set_bit_manual(uint32_t x)
{
    if (!x) return -1;
```

```
    int pos = 0;
    while ((x & 1U) == 0) {    /* while LSB is 0... */
        x >>= 1;               /* ...shift right (discard LSB) */
        pos++;                 /* ...and count the step */
    }
    return pos;                /* pos where (x >> pos) & 1 is first true */
}

/* isolation trick
 *
 * -x in two's complement = ~x + 1
 * ~x flips all bits, +1 then ripples carry rightward until it sets
 * exactly the position of the original lowest set bit.
 * Result is a VALUE with only that one bit set, not the position.
 *
 * e.g.  x      = 0000 1010 0  (LSB at bit 1)
 *       -x     = 1111 0110 0
 *       x & -x = 0000 0010 0  <-> isolated LSB as a value          */
uint32_t isolate_lsb(uint32_t x)
{
    return x & (-x);           /* returns the bit VALUE, not position */
}
```

x = 0000 1010 1000 (lowest set bit at position 3) x - 1 = 0000 1010 0111 (borrow flips bits 0..3) x & -x = 0000 0000 1000 (isolates ONLY bit 3 as a value)

3.5.9 Find last set bit in a number

```
/* Keep shifting x right until it becomes 0.
 * Each shift discards one bit from the top.
 * The number of shifts needed = position of the highest set bit.
 *
 * e.g.  x = 0110 0100  (decimal 100)
 *       shift 1 -> 0011 0010   pos = 1
 *       shift 2 -> 0001 1001   pos = 2
 *       ...
 *       shift 6 -> 0000 0001   pos = 6
 *       shift 7 -> 0000 0000   loop exits, return pos = 6           */
int last_set_bit_shift(uint32_t x)
{
    if (!x) return -1;

    int pos = 0;
    while(x) {
        x = x >> 1     /* shift right; loop exits when x becomes 0 */
        pos++;         /* count each shift, final value = MSB position */
    }
    return pos;
}

/* Binary search / divide-and-conquer
 *
 * Narrows down the MSB position in exactly 5 steps (log2 of 32).
```

```
 * Checks whether any bit exists in the upper half; if yes, shift
 * the entire value down by that half and accumulate the offset.
 * O(log 32) = O(5) - no loop dependency on the actual bit position.
 *
 * Step sizes:  16 -> 8 -> 4 -> 2 -> 1                                */
int last_set_bit_bsearch(uint32_t x)
{
    if (!x) return -1;

    int pos = 0;

    if (x & 0xFFFF0000U) { pos += 16; x >>= 16; }  /* bit in upper 16? */
    if (x & 0x0000FF00U) { pos +=  8; x >>=  8; }  /* bit in upper 8?  */
    if (x & 0x000000F0U) { pos +=  4; x >>=  4; }  /* bit in upper 4?  */
    if (x & 0x0000000CU) { pos +=  2; x >>=  2; }  /* bit in upper 2?  */
    if (x & 0x00000002U) { pos +=  1; x >>=  1; }  /* bit in upper 1?  */

    return pos;
}
```

3.5.10 Reverse bits of 32-bit number

```
/*
 * Reverse all 32 bits of an unsigned integer.
 * e.g.  0b 1011 0100 0000 0000 0000 0000 0000 0001
 *       0b 1000 0000 0000 0000 0000 0000 0010 1101 (result)
 */

/* Divide-and-conquer with bitmasks (portable, O(log 32))
 *
 * Core idea: repeatedly swap the two halves of the number,
 * each time at half the previous granularity.
 *
 * 5 stages: 16-bit halves -> bytes -> nibbles -> bit-pairs -> bits
 *
 * Mask anatomy (memorise these - they come up in other problems too):
 *
 *   0xFFFF0000 = 1111 1111 1111 1111 | 0000 0000 0000 0000  (upper half)
 *   0x0000FFFF = 0000 0000 0000 0000 | 1111 1111 1111 1111  (lower half)
 *
 *   0xFF00FF00 = 1111 1111 | 0000 0000 | 1111 1111 | 0000 0000  (odd bytes)
 *   0x00FF00FF = 0000 0000 | 1111 1111 | 0000 0000 | 1111 1111  (even bytes)
 *
 *   0xF0F0F0F0 = 1111 | 0000 | 1111 | 0000 | ...  (odd nibbles)
 *   0x0F0F0F0F = 0000 | 1111 | 0000 | 1111 | ...  (even nibbles)
 *
 *   0xCCCCCCCC = 11 | 00 | 11 | 00 | ...           (odd bit-pairs)
 *   0x33333333 = 00 | 11 | 00 | 11 | ...           (even bit-pairs)
 *
 *   0xAAAAAAAA = 1010 1010 ...                     (odd single bits)
 *   0x55555555 = 0101 0101 ...                     (even single bits)
 *
 * Pattern: each stage's two masks are bitwise complements of each other.
 * Each stage shifts by exactly half its working block size.          */

uint32_t reverse32(uint32_t x)
{
```

```
    /* Stage 1: swap upper 16 bits with lower 16 bits
     *   before: [A B C D | E F G H | I J K L | M N O P]
     *   after:  [I J K L | M N O P | A B C D | E F G H]  */
    x = ((x & 0xFFFF0000U) >> 16) |
        ((x & 0x0000FFFFU) << 16);

    /* Stage 2: swap odd bytes with even bytes within each 16-bit half
     *   before: [I J K L | M N O P | A B C D | E F G H]
     *   after:  [M N O P | I J K L | E F G H | A B C D]  */
    x = ((x & 0xFF00FF00U) >>  8) |
        ((x & 0x00FF00FFU) <<  8);

    /* Stage 3: swap odd nibbles with even nibbles within each byte
     *   each byte: [HI LO] -> [LO HI]                     */
    x = ((x & 0xF0F0F0F0U) >>  4) |
        ((x & 0x0F0F0F0FU) <<  4);

    /* Stage 4: swap odd bit-pairs with even bit-pairs within each nibble
     *   each nibble: [b3 b2 | b1 b0] -> [b1 b0 | b3 b2]   */
    x = ((x & 0xCCCCCCCCU) >>  2) |
        ((x & 0x33333333U) <<  2);

    /* Stage 5: swap every odd bit with its even neighbour
     *   final swap of adjacent bits — completes the full reversal  */
    x = ((x & 0xAAAAAAAAU) >>  1) |
        ((x & 0x55555555U) <<  1);

    return x;
}

/* Naive loop
 *
 * Extract LSB of x each iteration, push it into result from the MSB end.
 * O(32) — always 32 iterations regardless of bit pattern.          */
uint32_t reverse32_loop(uint32_t x)
{
    uint32_t result = 0;
    int bits = 32;

    while (bits--) {
        result = result << 1;        /* shift result up*/
        result = result | (x & 1U);  /* OR in LSB of x */
        x = x >> 1;                  /* discard the LSB just consumed */
    }

    return result;
}
```

3.5.11 Swap even and odd bits

```
uint32_t swap_even_odd_bits(uint32_t x)
{
    uint32_t odd_bits  = x & 0xAAAAAAAAU;   /* isolate bits at pos 1,3,5,7,...
                                               zero out all even-position bits  */

    uint32_t even_bits = x & 0x55555555U;   /* isolate bits at pos 0,2,4,6,...
                                               zero out all odd-position bits   */
```

```
    return (odd_bits  >> 1)                /* slide odd-position bits DOWN by 1
                                              pos 1->0, 3->2, 5->4, 7->6, ...     */
         | (even_bits << 1);               /* slide even-position bits UP by 1
                                              pos 0->1, 2->3, 4->5, 6->7, ...     */
}
```

3.5.12 Check if the number has only one bit set

If the number is power of 2 then that number has only one bit set.

3.5.13 Clear all the bits from MSB to nth bit pos

```
/*
 * Clear all bits from the MSB down to position n (inclusive).
 * Keep bits [n-1 .. 0] untouched.
 *
 * Visual:
 *   bit pos:  31 30 29 ... n+1  n  | n-1  n-2 ... 1   0
 *   before:    ?  ?  ? ...  ?   ?  |  ?    ?  ... ?   ?
 *   after:     0  0  0 ...  0   0  |  ?    ?  ... ?   ?   <- upper cleared
 *                                   ^^^^^^^^^^^^^^^^^^
 *                                   these bits preserved exactly
 *
 * e.g.  x = 0xFF  (1111 1111),  n = 4
 *       clear bits [7..4], keep bits [3..0]
 *       result = 0x0F  (0000 1111)
 */

/*  Method 1: Lower-bits mask (preferred – one expression)
 *
 * Key insight: (1U << n) - 1  produces exactly n ones in the low positions.
 *
 *   n = 4:
 *   1U << 4         = 0000 0000 0001 0000   (1 at position n)
 *   (1U << 4) - 1   = 0000 0000 0000 1111   (all ones below position n)
 *
 * AND-ing with this mask:
 *   – bits [n-1..0] : masked with 1 -> preserved unchanged
 *   – bits [31..n]  : masked with 0 -> forced to 0 (cleared)           */

uint32_t clear_msb_to_n(uint32_t x, uint8_t n)
{
    return x & ((1U << n) - 1U);    /* keep only the lower n bits */
    /*
     * (1U << n) - 1U  builds the "keep" mask at compile time if n is
     * a constant – zero runtime cost on any architecture.
     */
}

/*  Method 2: Complement + shift (shows understanding of bit width)
 *
 * Build a mask of ones for the upper bits, then invert and AND.
 *
 *   ~0U             = 1111 1111 1111 1111 1111 1111 1111 1111
 *   ~0U << n        = 1111 1111 1111 1111 1111 1111 1111 0000  (n=4)
 *   ~(~0U << n)     = 0000 0000 0000 0000 0000 0000 0000 1111  <- same mask
```

```
 *
 * Identical result to Method 1, but makes the complement relationship
 * explicit - useful when explaining to an interviewer.                 */

uint32_t clear_msb_to_n_v2(uint32_t x, uint8_t n)
{
    return x & ~(~0U << n);         /* ~0U = all ones; shift left clears low n;
                                       invert again -> low n ones = keep mask  */
}
```

3.5.14 Extract a bit field from a register

3.5.14.1 Given start bit position and len

```
uint32_t extract_bitfield(uint32_t reg, uint8_t start, uint8_t width)
{
    return (reg >> start)               /* Step 1: shift field to bit 0      */
         & ((1U << width) - 1U);        /* Step 2: mask off everything above */
}
```

3.5.14.2 Given start bit position and end bit position

```
uint32_t extract_bits(uint32_t value,
                      uint8_t start,
                      uint8_t end)
{
    uint32_t width = end - start + 1;
    uint32_t mask = (1U << width) - 1;
    return (value >> start) & mask;
}
```

3.5.14.3 Given mask and position

```
uint32_t extract_with_cmsis_style(uint32_t reg,
                                  uint32_t mask,
                                  uint8_t  pos)
{
    return (reg & mask) >> pos;
}
```

3.5.15 Insert a bit field into a register

3.5.15.1 Given start bit position and width

```
uint32_t insert_bitfield(uint32_t reg,
                         uint32_t field_val,
                         uint8_t start,
                         uint8_t width)
{
    uint32_t mask = ((1U << width) - 1U) << start;

    reg &= ~mask;                           /* Clear field region          */
    reg |= (field_val << start) & mask;     /* Insert new field value      */

    return reg;
}
```

3.5.15.2 Given start bit position and end bit position

```c
uint32_t insert_bits(uint32_t reg,
                     uint32_t field_val,
                     uint8_t start,
                     uint8_t end)
{
    uint32_t width = end - start + 1;
    uint32_t mask = ((1U << width) - 1U) << start;

    reg &= ~mask;                           /* Clear target bits           */
    reg |= (field_val << start) & mask;     /* Insert field                */

    return reg;
}
```

3.5.15.3 Given mask and position

```c
uint32_t insert_with_cmsis_style(uint32_t reg,
                                 uint32_t field_val,
                                 uint32_t mask,
                                 uint8_t pos)
{
    reg &= ~mask;                           /* Clear field                 */
    reg |= (field_val << pos) & mask;       /* Insert shifted value        */

    return reg;
}
```

3.6 Functions_1

3.7 Functions_2

3.8 Functions_3

3.9 Introdution_auto

3.10 Static_Storage_Class

3.11 Extern_Storage_Class

3.12 Compilation_Process

3.13 Extern MultiFile Programming

3.14 Extern MultiFile Programming - 1

3.15 MemorySegments

3.16 Register_Storage_Class

3.17 Intro to pointer

3.18 Operations allowed on Pointer

3.19 Sizeof Pointer

3.20 Little and Big Endian Architecture

3.21 void pointer

3.22 Wild and NULL Pointer

3.23 callbyvalue and callbyreferance

3.24 Dangling Pointer

3.25 Recursion on Pointers

3.26 Pointer to Pointer

3.27 comments

3.28 Byts Arrays

3.29 Byts Declarations

3.30 Byts Array

3.31 Byts Arrays 1

3.32 Byts 2D Ayyas

3.33 Byts 3D Arrays Revised 2